

Ок, тоді не просто список, а що саме воно робить і для чого потрібно.

1. Що ми будуємо

Ти робиш **Flight Storage Service** — Web API для роботи з рейсами.

Його задача:

```
зберігати рейси,  
шукати рейси,  
додавати рейси,  
оновлювати рейси,  
видаляти рейси.
```



Тобто це backend-сервіс, який приймає HTTP-запити і працює з базою даних.

2. Загальна схема проєкту

У тебе зараз така архітектура:

```
Client / Swagger / Postman  
    ↓  
FlightsController  
    ↓  
FlightService  
    ↓  
FlightRepository  
    ↓  
PostgreSQL
```



Це означає:

Postman відправляє запит.

Controller приймає цей запит.

Service перевіряє бізнес-логіку.

Repository виконує SQL-запит.

PostgreSQL зберігає або повертає дані.

3. Model — Flight

Модель — це клас, який описує один рейс.

</> C#



```
public class Flight
{
    public string FlightNumber { get; set; }
    public DateTime DepartureDateTime { get; set; }
    public string DepartureAirportCity { get; set; }
    public string ArrivalAirportCity { get; set; }
    public int DurationMinutes { get; set; }
}
```

Для чого вона потрібна

Щоб у програмі був один зрозумілий об'єкт "рейс".

Не просто окремо:

номер
дата
місто
тривалість



а один об'єкт:

</> C#



```
Flight flight
```

Що означають поля

FlightNumber

Номер рейсу.

Наприклад:

PS777



Використовується для пошуку, оновлення і видалення.

DepartureDateTime

Дата і час вильоту.

Наприклад:

2026-06-20T14:30:00



Потрібно для пошуку рейсів по даті.

DepartureAirportCity

Місто відправлення.

Наприклад:

Kyiv



ArrivalAirportCity

Місто прибуття.

Наприклад:

Rome



DurationMinutes

Тривалість рейсу в хвиликах.

Наприклад:

150



4. Controller — FlightsController

Controller — це точка входу в API.

Він відповідає на питання:

Який HTTP-запит прийшов?

Який метод сервісу викликати?

Який HTTP-статус повернути?



Controller не має сам працювати з базою.

Погано:

```
</> C#
```



```
Controller → PostgreSQL
```

Правильно:

```
</> C#
```



```
Controller → Service → Repository → PostgreSQL
```

```
[ApiController]
```

```
</> C#
```



```
[ApiController]
```

Каже ASP.NET, що цей клас є API-контролером.

Він допомагає автоматично обробляти HTTP-запити, параметри, тіло запиту, помилки моделі.

```
[Route("api/[controller]")]
```

```
</> C#
```



```
[Route("api/[controller]")]
```

Це маршрут контролера.

Назва класу:

```
</> C#
```



```
FlightsController
```

ASP.NET прибирає слово `Controller` і отримує:

```
flights
```



Тому базовий шлях:

```
</> http
```



```
/api/flights
```

5. Endpoint-и

Endpoint — це конкретний URL + HTTP-метод.

GET by number

```
</> http
```



```
GET /api/flights/PS777
```

Метод:

```
</> C#
```



```
[HttpGet("{flightNumber}")]
public IActionResult GetByNumber(string flightNumber)
```

Що робить

Шукає один рейс по номеру.

Якщо рейс є:

```
</> http
```



```
200 OK
```

Якщо рейсу нема:

```
</> http
```



```
404 Not Found
```

Якщо номер пустий:

```
</> http
```



```
400 Bad Request
```

GET by date

</> http



```
GET /api/flights?date=2026-06-20
```

Метод:

</> C#



```
[HttpGet]  
public IActionResult GetByDate(DateTime date)
```

Що робить

Повертає всі рейси на конкретну дату.

Тобто шукає по даті вильоту.

GET departure

</> http



```
GET /api/flights/departure?city=Kyiv&date=2026-06-20
```

Метод:

</> C#



```
[HttpGet("departure")]  
public IActionResult GetByDeparture(string city, DateTime date)
```

Що робить

Шукає всі рейси, які вилітають з певного міста в певну дату.

GET arrival

</> http



```
GET /api/flights/arrival?city=Rome&date=2026-06-20
```

Метод:

</> C#



```
[HttpGet("arrival")]  
public IActionResult GetByArrival(string city, DateTime date)
```

Що робить

Шукає всі рейси, які прилітають у певне місто в певну дату.

GET all

</> http



```
GET /api/flights/all
```

Метод:

</> C#



```
[HttpGet("all")]  
public IActionResult GetAll()
```

Що робить

Повертає всі рейси з бази.

POST

</> http



```
POST /api/flights
```

Метод:

</> C#



```
[HttpPost]  
public IActionResult AddFlight([FromBody] Flight flight)
```

Що робить

Створює новий рейс.

[FromBody] означає, що дані беруться з JSON body.

Наприклад:

</> JSON



```
{
  "flightNumber": "PS777",
  "departureDateTime": "2026-06-20T14:30:00",
  "departureAirportCity": "Kyiv",
  "arrivalAirportCity": "Rome",
  "durationMinutes": 150
}
```

Якщо створено:

</> http



201 Created

PUT

</> http



PUT /api/flights/PS777

Метод:

</> C#



```
[HttpPut("{flightNumber}")]
public IActionResult UpdateFlight(string flightNumber, [FromBody] Flight flight)
```

Що робить

Оновлює існуючий рейс.

Номер рейсу береться з URL:

PS777



А нові дані беруться з body.

Наприклад:

JSON



```
{
  "departureDateTime": "2026-06-21T10:00:00",
  "departureAirportCity": "Kyiv",
  "arrivalAirportCity": "Paris",
  "durationMinutes": 170
}
```

Якщо оновлено:

http



204 No Content

Чому не 200 OK?

Бо ми не повертаємо тіло відповіді, просто кажемо:

оновлення успішне



DELETE

http



DELETE /api/flights/PS777

Метод:

C#



```
[HttpDelete("{flightNumber}")]
public IActionResult DeleteFlight(string flightNumber)
```

Що робить

Видаляє рейс по номеру.

Якщо видалено:

http



204 No Content

Якщо не знайдено:

```
</> http
```



```
404 Not Found
```

6. IActionResult

```
</> C#
```



```
public IActionResult GetByNumber(...)
```

IActionResult означає:

цей метод може повернути різні HTTP-відповіді



Наприклад:

```
</> C#
```



```
return Ok(flight);  
return NotFound();  
return BadRequest(exception.Message);  
return NoContent();  
return CreatedAtAction(...);
```

Тому для контролера це зручно.

7. HTTP статуси

Ok ()

```
</> C#
```



```
return Ok(flight);
```

Повертає:

```
</> http
```



```
200 OK
```

Коли все добре і є дані.

NotFound()

</> C#



```
return NotFound();
```

Повертає:

</> http



404 Not Found

Коли запис не знайдено.

BadRequest()

</> C#



```
return BadRequest(exception.Message);
```

Повертає:

</> http



400 Bad Request

Коли клієнт передав неправильні дані.

Наприклад:

```
порожній flightNumber  
порожнє city  
durationMinutes <= 0
```



CreatedAtAction()

</> C#



```
return CreatedAtAction(  
    nameof(GetByNumber),  
    new { flightNumber = flight.FlightNumber },  
    flight);
```

Повертає:

```
</> http
```



```
201 Created
```

Це правильна відповідь для POST.

Вона каже:

```
ресурс створено  
ось де його можна знайти
```



```
NoContent()
```

```
</> C#
```



```
return NoContent();
```

Повертає:

```
</> http
```



```
204 No Content
```

Означає:

```
операція успішна, але тіло відповіді порожнє
```



Добре підходить для PUT і DELETE.

8. Service — FlightService

Service — це шар бізнес-логіки.

Він відповідає на питання:

```
Чи можна виконувати цю дію?  
Чи правильні дані?  
Що потрібно перевірити перед зверненням до бази?
```



Service не повинен знати деталей SQL.

Він не пише:

```
</> SQL
```



```
SELECT * FROM flights
```

Він просто викликає repository:

```
</> C#
```



```
_flightRepository.GetByNumber(flightNumber);
```

GetByNumber

```
</> C#
```



```
public Flight? GetByNumber(string flightNumber)
```

Що робить

Перевіряє, чи номер рейсу не пустий.

```
</> C#
```



```
if (string.IsNullOrEmpty(flightNumber))
```

Якщо пустий — кидає помилку.

Потім звертається до repository.

Flight?

```
</> C#
```



```
public Flight? GetByNumber(...)
```

Знак `?` означає:

```
метод може повернути Flight або null
```



Чому?

Бо рейсу з таким номером може не бути.

GetByDate

</> C#



```
public List<Flight> GetByDate(DateTime date)
```

Що робить

Перевіряє, що дата не є дефолтною.

</> C#



```
if (date == default)
```

Потім повертає список рейсів.

List<Flight>

Означає:

список рейсів



Бо на одну дату може бути багато рейсів.

GetByDepartureCityAndDate

Перевіряє, що місто відправлення не пусте.

Потім повертає всі рейси з цього міста на дату.

GetByArrivalCityAndDate

Перевіряє, що місто прибуття не пусте.

Потім повертає всі рейси в це місто на дату.

AddFlight

</> C#



```
public void AddFlight(Flight flight)
{
    ValidateFlight(flight, requireFlightNumber: true);

    _flightRepository.AddFlight(flight);
}
```

Що робить

Перед додаванням рейсу перевіряє всі важливі поля:

```
flight не null
flightNumber не пустий
departure city не пусте
arrival city не пусте
departure date є
duration > 0
```

Після цього викликає repository.

UpdateFlight

</> C#

```
public void UpdateFlight(string flightNumber, Flight flight)
```

Що робить

1. Перевіряє номер рейсу з URL.
2. Перевіряє нові дані рейсу.
3. Перевіряє, чи рейс існує.
4. Якщо існує — оновлює.

Ось важлива частина:

</> C#

```
var existingFlight = _flightRepository.GetByNumber(flightNumber);

if (existingFlight == null)
{
    throw new KeyNotFoundException("Flight not found.");
}
```

Для чого це

Щоб не вийшло так:

```
PUT /api/flights/NO_SUCH_FLIGHT
```



і API сказав:

```
<> http
```



```
204 No Content
```

хоча реально нічого не оновилося.

DeleteFlight

```
<> C#
```



```
public void DeleteFlight(string flightNumber)
```

Що робить

1. Перевіряє номер.
2. Перевіряє, чи рейс існує.
3. Видаляє.

Це правильніше, ніж просто одразу робити DELETE.

9. ValidateFlight

```
<> C#
```



```
private static void ValidateFlight(Flight flight, bool requireFlightNumber)
```

Це приватний метод для валідації рейсу.

Для чого ми його винесли

Щоб не дублювати один і той самий код у:

```
<> C#
```



```
AddFlight  
UpdateFlight
```

Це принцип:

DRY — Don't Repeat Yourself



`private`

⟨⟩ C#



```
private static void ValidateFlight(...)
```

`private` означає:

метод доступний тільки всередині `FlightService`



Ззовні його викликати не можна.

`static`

⟨⟩ C#



```
private static void ValidateFlight(...)
```

`static` тут означає, що метод не використовує поля класу.

Він не використовує:

⟨⟩ C#



```
_flightRepository
```

Тому може бути `static`.

`requireFlightNumber`

⟨⟩ C#



```
ValidateFlight(flight, requireFlightNumber: true);
```

Для POST номер рейсу обов'язковий у body.

POST /api/flights



Там треба створити новий рейс, тому номер має бути в JSON.

```
</> C#
```



```
ValidateFlight(flight, requireFlightNumber: false);
```

Для PUT номер рейсу береться з URL:

```
</> http
```



```
PUT /api/flights/PS777
```

Тому в body номер можна не передавати.

10. Repository — FlightRepository

Repository — це шар доступу до даних.

Він відповідає на питання:

Який SQL-запит виконати?

Як відкрити з'єднання з БД?

Як прочитати результат?

Як перетворити рядок БД у Flight?



Repository не має вирішувати бізнес-правила.

Наприклад, не його задача думати:

```
чи duration > 0
```



Це задача Service.

11. Npgsql

```
</> C#
```



```
using Npgsql;
```

Npgsql — це бібліотека для роботи з PostgreSQL у .NET.

Вона дає класи:

</> C#



```
NpgsqlConnection  
NpgsqlCommand  
NpgsqlDataReader
```

12. Connection string

</> C#



```
private readonly string _connectionString;
```

Connection string — це рядок підключення до бази.

У ньому є:

```
host  
port  
database  
username  
password
```



Repository бере його з:

</> C#



```
configuration.GetConnectionString("FlightsDb")
```

Для чого це

Щоб не писати пароль і адресу бази прямо в коді.

Правильно зберігати це в:

```
appsettings.json
```



13. NpgsqlConnection

</> C#



```
using var connection = new NpgsqlConnection(_connectionString);  
connection.Open();
```

Це підключення до PostgreSQL.

Що робить

Відкриває канал між твоїм API і базою даних.

```
using var
```

</> C#



```
using var connection = ...
```

Означає:

після завершення методу ресурс автоматично закриється



Це важливо, бо підключення до БД треба закривати.

14. NpgsqlCommand

</> C#



```
using var command = new NpgsqlCommand(sql, connection);
```

Це SQL-команда, яку ти хочеш виконати.

Наприклад:

</> SQL



```
SELECT ...  
INSERT ...  
UPDATE ...  
DELETE ...
```

15. SQL параметри

</> C#



```
command.Parameters.AddWithValue("@flightNumber", flightNumber);
```

Для чого це

Щоб безпечно передати значення в SQL.

Правильно:

</> SQL



```
WHERE flight_number = @flightNumber
```

Погано:

</> C#



```
"WHERE flight_number = '" + flightNumber + "'"
```

Бо це може привести до SQL Injection.

16. ExecuteReader

</> C#



```
using var reader = command.ExecuteReader();
```

Використовується для SELECT.

Тобто коли ми очікуємо отримати дані з бази.

17. ExecuteNonQuery

</> C#



```
command.ExecuteNonQuery();
```

Використовується для команд, які не повертають таблицю:

</> SQL



```
INSERT
UPDATE
DELETE
```

18. MapFlight

</> C#



```
private static Flight MapFlight(NpgsqlDataReader reader)
```

Що робить

Бере один рядок з бази і перетворює його в об'єкт `Flight`.

</> C#



```
FlightNumber = reader.GetString(0)
```

Означає:

взяти першу колонку з SELECT



Тому порядок колонок у SELECT важливий.

19. SQL методи

GetByNumber

</> SQL



```
SELECT ...
FROM flights
WHERE flight_number = @flightNumber;
```

Шукає один рейс по номеру.

GetByDate

</> SQL



```
WHERE departure_datetime::date = @date
```

Тут:

SQL



```
departure_datetime::date
```

означає:

взяти тільки дату без часу



Бо в базі може бути:

2026-06-20 14:30:00



А ти шукаєш:

2026-06-20



GetByDepartureCityAndDate

SQL



```
WHERE departure_airport_city = @city  
AND departure_datetime::date = @date;
```

Шукає рейси за містом вильоту і датою.

GetByArrivalCityAndDate

SQL



```
WHERE arrival_airport_city = @city  
AND departure_datetime::date = @date;
```

Шукає рейси за містом прибуття і датою вильоту.

GetAll

SQL



```
SELECT ...  
FROM flights  
ORDER BY departure_datetime;
```

Повертає всі рейси.

`ORDER BY departure_datetime` сортує їх за датою вильоту.

Це не фільтр.

Воно не змінює, які рейси повернуться.

Воно змінює тільки порядок.

AddFlight

SQL



```
INSERT INTO flights (...)  
VALUES (...);
```

Додає новий запис у таблицю.

UpdateFlight

SQL



```
UPDATE flights  
SET ...  
WHERE flight_number = @flightNumber;
```

Оновлює рейс по номеру.

Ти правильно зробив, що номер береться з параметра методу:

C#



```
UpdateFlight(string flightNumber, Flight flight)
```

а не з body.

DeleteFlight

</> SQL



```
DELETE FROM flights
WHERE flight_number = @flightNumber;
```

Видаляє рейс по номеру.

20. Interfaces

У тебе є:

</> C#



```
IFlightService
IFlightRepository
```

Для чого вони потрібні

Щоб класи залежали не від конкретної реалізації, а від контракту.

Наприклад, Controller знає тільки:

</> C#



```
IFlightService
```

Йому не важливо, який саме клас там підставлений.

Це дає гнучкість.

Сьогодні в тебе:

```
FlightRepository через Npgsql
```



Пізніше можна зробити:

```
FlightRepository через Dapper
```



А Controller і Service майже не зміняться.

21. Dependency Injection

У `Program.cs` має бути щось типу:

</> C#



```
builder.Services.AddScoped<IFlightService, FlightService>();  
builder.Services.AddScoped<IFlightRepository, FlightRepository>();
```

Що це означає

Коли ASP.NET бачить:

</> C#



```
public FlightsController(IFlightService flightService)
```

він думає:

Ага, потрібен IFlightService.
Я знаю, що для IFlightService треба створити FlightService.



А коли створює `FlightService`, бачить:

</> C#



```
public FlightService(IFlightRepository flightRepository)
```

і створює `FlightRepository`.

22. Postman

Postman потрібен для тестування API.

Ти можеш вручну перевірити:

</> http



```
GET  
POST  
PUT  
DELETE
```

Без фронтенду.

23. Повний набір запитів

POST

</> http



POST /api/flights

Створює рейс.

GET by number

</> http



GET /api/flights/PS777

Перевіряє, чи рейс створився.

GET by date

</> http



GET /api/flights?date=2026-06-20

Шукає рейси на дату.

GET departure

</> http



GET /api/flights/departure?city=Kyiv&date=2026-06-20

Шукає рейси з міста.

GET arrival

</> http



GET /api/flights/arrival?city=Rome&date=2026-06-20

Шукає рейси в місто.

GET all

</> http



GET /api/flights/all

Повертає всі рейси.

PUT

</> http



PUT /api/flights/PS777

Оновлює рейс.

DELETE

</> http



DELETE /api/flights/PS777

Видаляє рейс.

24. Що конкретно є в проєкті зараз

У тебе є:

Models
Controllers
Services
Repositories
Interfaces
PostgreSQL database
Swagger
Postman testing



Models

```
Flight.cs
```



Описує рейс.

Services

```
IFlightService.cs
```

```
FlightService.cs
```



Валідація і бізнес-логіка.

Repositories

```
IFlightRepository.cs
```

```
FlightRepository.cs
```



Робота з PostgreSQL.

Controllers

```
FlightsController.cs
```



HTTP endpoint-и.

Database

Таблиця:

```
flights
```



Колонки:

```
flight_number
```

```
departure_datetime
```

```
departure_airport_city
```

```
arrival_airport_city
```

```
duration_minutes
```



25. Що ти реально сьогодні зрозумів

Найважливіше не те, що ти просто написав CRUD.

А те, що ти почав розуміти:

де має бути яка відповідальність



Controller не має валідувати всю бізнес-логіку.

Repository не має думати про правила.

Service не має писати SQL.

Кожен шар має свою задачу.

Це і є нормальна backend-архітектура.